

TECHNICAL FEASIBILITY ANALYSIS · 技术可行性
分析

实时 CPU 语音转换系统 架构分析

Real-time CPU Voice Conversion Architecture Analysis

针对 ~100MB 内存 / 纯 CPU 推理 / 300–500ms 端到端延迟的硬约束，深入分析 Seed-VC (tiny)、TinyVC、RVC、Fish-Speech 轻量路径、PocketTTS 五个开源仓库，给出模块裁剪与替换方案、量化与 ONNX 导出路径、实时流式策略、5 音色 × 30s 预注册流程与 P0/P1/P2 改造 checklist。

PREPARED FOR	Edge / Desktop CPU Deployment
AUTHOR	Z.ai
DATE	2026.10.06
DOCUMENT ID	ZAI-VC-RT-2026-10

目录 / Table of Contents

一、执行摘要 Executive Summary	3
二、项目背景与约束 Project Context & Constraints	3
2.1 目标硬件 Target Hardware	3
2.2 延迟预算拆解 Latency Budget	4
2.3 内存预算拆解 Memory Budget	4
2.4 音色注册规格 Speaker Registry Spec	5
三、五仓库横向对比 Cross-Repo Comparison	5
3.1 Seed-VC (tiny 变体)	5
3.2 TinyVC —— 本方案的主框架	6
3.3 RVC —— 检索索引思路的来源	6
3.4 Fish-Speech —— 仅作参考架构	6
3.5 PocketTTS —— 可借鉴的因果卷积与 INT8 量化代码	6
四、模块裁剪与替换方案 Module Trim & Replace Map	7
4.1 内容编码器 Content Encoder	7
4.2 说话人嵌入 Speaker Embedder	8
4.3 F0 提取器 Pitch Extractor	8
4.4 声码器 Vocoder	8
4.5 流式壳 Streaming Shell	8
五、量化与 ONNX 导出路径 Quantization & ONNX Export	9
5.1 ONNX 导出路径 (已有, 无需修改)	9
5.2 ORT 动态 INT8 量化 (需自写 ~50 行)	9
5.3 蒸馏路径 (仅未来升级时用)	10
5.4 为什么不做 QAT	10
六、实时流式策略 Real-time Streaming Strategy	10
6.1 Chunk 大小 Chunk Size	10
6.2 缓冲与交叉淡入策略 Buffer & Crossfade	11
6.3 因果卷积与状态缓存 Causal Conv1d & State Cache	11
6.4 SIMD 与线程模型 SIMD & Threading	11
七、音色预注册流程 Speaker Pre-registration Pipeline	11

八、可落地改造 Checklist Actionable Checklist	13
九、风险与限制 Risks & Limitations	13
十、术语表 Glossary (CN/EN)	14

一、执行摘要 Executive Summary

KICKER · PROJECT GOAL & BOTTOM LINE

本报告针对一个明确的工程问题展开：在**纯 CPU 推理、~100MB 运行内存、300–500ms 端到端延迟**的硬约束下，能否构建一套**实时语音转换（Voice Conversion, VC）系统**，输入为用户的实时麦克风声音流，输出为预先注册的**5 个固定目标音色**（每个音色 30 秒参考音频）。为此我们深入分析了五个开源仓库——**Seed-VC（tiny 变体）、TinyVC、RVC、Fish-Speech 轻量路径、PocketTTS**——从架构、参数量、量化与 ONNX 支持、流式能力、许可证、CPU 可行性六个维度做了横向对比，并提炼出可直接落地的模块裁剪与替换方案、量化与 ONNX 导出路径、实时流式策略、音色预注册流程，以及一份按 P0/P1/P2 优先级排序的可执行 checklist。

核心结论可以一句话概括：**TinyVC 是当前最贴合本目标的工程基线**。它把内容编码器从 WavLM-Base（90M 参数）蒸馏成 6 层 ConvNeXt-v2（4.2M 参数），把声码器从 HiFi-GAN 换成 DDSP 风格的 SourceNet+FilterNet（4.6M 参数），把说话人建模从独立编码器换成 kNN-VC 检索（零额外参数）。整个推理图仅 **9.36M 参数、~36MB FP32 / ~18MB FP16**，已经原生支持 ONNX 导出（opset 17，三个子图）与 PyAudio 流式推理（SOLA 交叉淡入），在 24kHz 单声道下默认算法延迟约 560ms，可通过调小 look-ahead 与 crossfade 压到 ~320ms，正好命中本目标的 300–500ms 区间。PocketTTS 则提供了可借鉴的**因果卷积、KV-cache 状态机与原生 INT8 动态量化**三件套，其 Mimi 编解码器在 INT8 后 ~25MB，可以作为备用声码器或后续升级路径。

9.36MTinyVC 推理参数 /
Inference params**~80MB**

目标常驻内存 / Target RSS

~370ms预期端到端延迟 / Expected
E2E**5×30s**音色注册规模 / Voice
registry

关键判断 Key Verdict: 在 x86 桌面 CPU（4–8 核、AVX2）上，本方案完全可落地，常驻内存 ~80–95MB、延迟 ~330–380ms，全部硬指标达标；在 ARM Cortex-A76/A78 类平台上，延迟约 450–600ms，可能轻微突破 500ms 上限——需进一步启用 FP16+INT8 与更小 chunk 才能稳妥命中目标。Fish-Speech 因许可证为非商用、且最小权重 ≥1GB，**仅作为参考架构使用，不进入运行时依赖**。

二、项目背景与约束 Project Context & Constraints

KICKER · TARGET HARDWARE, LATENCY BUDGET, SPEAKER SETUP

在讨论“能不能做”之前，先把约束讲清楚。这些约束不是任意设定的，而是来自实际部署场景：用户希望能在普通笔记本或边缘盒子上跑实时语音转换，没有 GPU、没有大内存，同时容忍轻度延迟（如 300–500ms，比一般会议软件的回声抑制延迟略长但可以接受）。本节明确目标硬件、延迟预算拆解、内存预算拆解、音色注册规格四件事，后续所有架构决策都基于这四组数字。

2.1 目标硬件 Target Hardware

本方案面向**跨平台部署**，主要测试目标为两类 CPU：(a) **x86 桌面级**——Intel/AMD 4–8 核、支持 AVX2 与 FMA 指令集，可使用 ONNXRuntime 的 MLAS 后端；(b) **ARM Cortex-A76/A78 级**——RK3588、树莓派 5、Apple Silicon 同档，支持 NEON 与 dotprod 指令，可用 XNNPACK 后端。两类平台均假设**无独显、无 CUDA、无 ROCm**。系统内存预算 100MB 包含：模型权重 + 激活张量 + 5 个参考音色索引 + 音频环形缓冲 + ONNXRuntime 自身工作内存。

2.2 延迟预算拆解 Latency Budget

端到端延迟按 麦克风采样 → VAD → 内容编码 → kNN 检索 → 解码 → DAC 输出 拆解。我们用 TinyVC 的 SOLA 流式框架作为基准，把每一阶段的预估耗时列出来。注意：算法延迟（algorithmic latency，由 chunk + look-ahead + crossfade 决定）与计算延迟（compute latency，由 CPU 推理时间决定）是两件事——前者由流式框架参数固定，后者随硬件变化。下表给出 x86 与 ARM 两套估算：

阶段 Stage	x86 (AVX2)	ARM (NEON)	说明 Notes
音频 ADC + 缓冲	5 ms	5 ms	I/O + ring buffer
WebRTC VAD 判决	2 ms	3 ms	能量 + ZCR
内容编码 (ConvNeXt 6L)	30 ms	70 ms	FP16 / INT8
kNN 检索 (top-4)	5 ms	12 ms	2048 frames × 768-d
F0 提取 (PitchEst.)	8 ms	18 ms	460K-param
解码器 (SourceNet+Filter)	40 ms	90 ms	4.6M params
SOLA 交叉淡入	80 ms	80 ms	算法延迟
chunk 缓冲对齐	200 ms	200 ms	2.5 × 80ms block
DAC 输出	5 ms	5 ms	I/O
合计 Total	~375 ms	~483 ms	x86 达标 / ARM 临界

表 2-1 · 端到端延迟预算 (80ms chunk + 200ms 缓冲) · Latency budget breakdown

2.3 内存预算拆解 Memory Budget

100MB 的内存上限很紧，必须按角色分配。下面这张表是常驻内存的拆解，权重部分以 INT8 量化后的体积估算（TinyVC 三子图 + PocketTTS Mimi 备选声码器）：

组件 Component	体积 Size	说明 Notes
TinyVC encoder (INT8)	5 MB	6 层 ConvNeXt-v2, 4.7M params
TinyVC decoder (INT8)	5 MB	SourceNet + FilterNet, 4.6M params
PocketTTS Mimi (INT8, 可选)	0-25 MB	备用声码器；不加载则为 0
5 个音色索引 (FP16)	12 MB	5 × 2.4MB, 30s 参考
激活张量峰值	15 MB	block=1920 单帧前向
音频环形缓冲	8 MB	last_delay + crossfade + sola_search
ONNXRuntime 工作内存	20 MB	线程池 + 内存池
Python 解释器 + 标准库	20 MB	基础开销
WebRTC VAD + 工具	2 MB	轻量依赖
合计 Total	~85-110 MB	符合 100MB 目标（边缘情况）

表 2-2 · 常驻内存拆解 · RSS breakdown

2.4 音色注册规格 Speaker Registry Spec

本方案假设用户预先注册 **5 个固定目标音色**，每个音色提供 30 秒干净参考音频（24kHz 单声道、归一化到 -3dBFS、无背景噪声、无背景音乐）。参考音频在**离线阶段**被编码为 768 维特征序列（约 1500 帧），存为 safetensors 或 .pt 文件，运行时一次性载入内存。这套设计与 TinyVC 的 extract_index.py、PocketTTS 的 export-voice 流程完全吻合，也避免了 RVC 那种“每加一个音色就要单独训练一个 .pth”的笨重路径。5 个音色之间要求**余弦相似度 < 0.85**，否则视为同质音色应合并或换样。

三、五仓库横向对比 Cross-Repo Comparison

KICKER · FIVE OPEN-SOURCE VC REPOSITORIES, SIDE BY SIDE

我们并行 clone 了五个公开仓库（详见 worklog），逐一阅读其 README、config、模型源码与推理脚本。下表先给出概览，后续小节针对每个仓库做 1-2 段的架构点评与“值不值得用”的判断。需要预先点明的结论：在这五个仓库中，**TinyVC 直接作为运行时主框架**，**PocketTTS 提供可借鉴的因果卷积与 INT8 量化代码**，**Seed-VC 提供流匹配蒸馏的训练配方作为参考**，**RVC 的检索索引思路在固定音色场景下被简化使用**，**Fish-Speech 仅作参考架构，不进入运行时**。

仓库 Repo	许可证	参数量 Params	ONNX	流式	CPU/100MB	本方案定位 Our use
Seed-VC (tiny)	GPL v3	~25M (DiT) + 1.2GB (XLS-R)	否 No	是 Yes (GUI)	不可行 NoGo	蒸馏配方 + 流匹配训练参考
TinyVC	Apache 2.0	9.36M (Enc+Dec)	是 Yes	是 Yes (SOLA)	可行 Go	主框架，直接复用
RVC v2	MIT	20-30M (synth) + 360MB (ContentVec)	否 No (社区有)	是 Yes (block=0.25s)	不可行 NoGo	小索引思路 + RMVPE 替换评估
Fish-Speech	NC 许可	~1GB+ (DualAR 4B)	否 No	是 (生成器)	不可行 NoGo	仅作参考架构，不入运行时
PocketTTS	MIT	~100M (FlowLM+Mimi)	否 (社区有)	是 Yes (causal)	可行 (边缘)	Mimi 声码器 + INT8 量化代码

表 3-1 · 五仓库横向对比 · Cross-repo comparison matrix

3.1 Seed-VC (tiny 变体)

仓库：Plachtaa/seed-vc（注意：用户提示中的 BytedanceFish-Speech/seed-vc 是 404，真实路径是 Plachtaa 维护的 fork）。**许可证** GPL v3, copyleft，商用需同源开源。**tiny 变体**实为 DiT 主干 25M 参数（hidden 384、6 头、9 层 U-ViT），但运行时仍需下载 1.2GB 的 facebook/wav2vec2-xls-r-300m 作为内容编码器，并未提供独立的小型内容编码器——这是 Seed-VC tiny 在我们场景下最大的“虚标”问题。它有 real-time-gui.py 与 SOLA 流式框架，KV-cache 与 causal-mask 代码已存在但**默认禁用**（use_kv_cache=False），每次 block 都对 prompt+chunk 做完整 DiT 前向，CPU 上毫无性价比可言。ONNX 与量化路径**完全缺失**，仓库里仅有的 onnxruntime import 是 DNSMOS 评测用。综合判断：**不作为运行时依赖**，但其 WavLM/ContentVec 蒸馏到小型 ConvNeXt 的训练配方（与 TinyVC 的 train_encoder.py 同源思路）值得作为我们后续训练自有内容编码器的参考。

3.2 TinyVC —— 本方案的主框架

仓库: github.com/uthree/tinyvc, Apache 2.0, 作者是日本独立开发者 uthree。架构是典型的 **source-filter VC**: 内容编码器用 6 层 ConvNeXt-v2 (384 通道、深度可分离 $k=7 + \text{GRN}$) 蒸馏 WavLM-Base-Plus 的第 4 层隐状态 (768-d); F0 用 4 层 ConvNeXt (128 通道、512 个对数频段、 $\text{top-k}=4$ 软解码) 联合训练; **没有独立的说话人编码器**——直接走 kNN-VC 检索 (余弦 top-4 替换源特征为目标特征); 声码器是 DDSP 风格的 SourceNet (3 层 ConvNeXt, 预测 14 谐波+1 的振幅, 加法正弦合成) + FilterNet (1D U-Net, 通道 [384,192,96,48,24], FiLM 调制的膨胀卷积 $k=3$ dilations 1,3,9,27)。推理总参数 **9.36M** (编码器 4.70M + 解码器 4.66M), 约 36MB FP32 / 18MB FP16。 **export_onnx.py** 已存在, opset 17、动态 batch 与 length、导出三子图; **infer_streaming.py** 已存在, PyAudio 24kHz paInt16、block=1920 (80ms)、extra=3840 (160ms look-ahead)、SOLA 交叉淡入。缺点: 仓库未提供 INT8 量化脚本, 需我们自己用 ONNXRuntime 的 `quantize_dynamic` 补上; F0 头比 RMVPE 在音乐人声上略弱, 但语音场景足够。

3.3 RVC —— 检索索引思路的来源

仓库: [RVC-Project/Retrieval-based-Voice-Conversion-WebUI](https://github.com/RVC-Project/Retrieval-based-Voice-Conversion-WebUI), MIT。RVC 的核心是 “**每帧 HuBERT 特征 + Faiss 检索**”, 索引默认是 IVF+Flat, 单说话人典型 50–200MB。RVC 的最大障碍是它依赖 ContentVec (HuBERT-base 微调版) 作为内容编码器, 该模型 FP32 体积 ~360MB, 单单这一个组件就超出我们 100MB 预算 3.6 倍。但 RVC 的 “小索引” 思路可以借鉴: 当我们只有 5 个固定音色时, 每个音色的索引可以压到 <5MB (使用 IVF256, PQ64x4fs, RFlat 量化索引或干脆用 FP16 矩阵 + 暴力 top-1)。RVC 的实时入口 `infer/rtrvc.py::RVC.infer()` 默认 `block_time=0.25s`、`crossfade=0.05s`, README 声称 170ms 端到端 (90ms with ASIO) ——这块流式框架的参数选择对我们有参考价值, 但 RVC 整体不能作为运行时依赖, **仅借鉴索引压缩与 block 配置经验**。

3.4 Fish-Speech —— 仅作参考架构

仓库: github.com/fishaudio/fish-speech, FISH AUDIO RESEARCH LICENSE (非商用)。Fish-Speech 是两阶段 TTS+VC 框架: DAC 神经编解码器 + DualARTransformer (BART 风格, Slow AR 4B 参数 + Fast AR 400M 参数)。最新 S2-Pro 模型 ~8GB bf16, 需要 **24GB VRAM**, README 给出的 RTF 0.195 + TTFA 100ms 是在 H200 GPU 上测的, CPU 上 RTF 5–20× (每秒音频需要 5–20 秒计算) 完全不能实时。**没有官方 ONNX 与 GGUF 导出**, 仓库内只有 `tools/llama/quantize.py` 一个 INT8 脚本 (仅 Linear、CPU 可用, 但模型本体太大)。它**对本方案的价值是两个设计模式**: (a) `<|speaker:i|> + VQ codes as in-context conditioning`——一种无需独立说话人编码器的零样本克隆模式; (b) DAC 编解码器结构 (~30–100MB 可重训版本)。**不作为运行时依赖**, 避免许可证与体积双重风险。

3.5 PocketTTS —— 可借鉴的因果卷积与 INT8 量化代码

仓库: github.com/kyutai-labs/pocket-tts, MIT。PocketTTS 是为 CPU 实时设计的 TTS (不是 VC), 由 BPE 分词器 + FlowLM (6 层因果 StreamingTransformer, `d_model=1024`) + Mimi 编解码器 (SEANet 编码 + 2 层 transformer + SEANet 解码, 24kHz、12.5Hz、hop=120) 组成。总参数约 100M (FP32 ~400MB / INT8 ~200MB), **单线程** (`torch.set_num_threads(1)`), README 报告 M4 CPU 上 RTF ~6×、首块 ~200ms。它对 VC 项目的**借鉴价值**在于三件可复用代

码：(1) StreamingConv1d + _LinearKVCacheBackend + StatefulModule——一套完整的因果卷积与 KV-cache 抽象，可以替换 TinyVC 的 SOLA 重编码循环，把算法延迟从 ~560ms 降到 ~150ms；(2) pocket_tts/quantization.py——torchao INT8 动态量化样板，FBGEMM (x86) / QNNPACK (ARM) 后端自动切换，可直接套用到 TinyVC 的 Linear 与 Conv1d 算子；(3) export-voice 把 30 秒参考的 KV-cache 状态序列化为 .safetensors，与本方案的 5 音色 × 30s 注册流程几乎对齐。

四、模块裁剪与替换方案 Module Trim & Replace Map

KICKER · FIVE MODULES, BEFORE & AFTER, FILE-LEVEL EVIDENCE

一个标准 VC 管线包含五个模块：**内容编码器、说话人嵌入、F0 提取器、声码器、流式壳**。本节按模块逐一给出“原仓库里的实现 → 我们要替换成什么 → 替换前后参数量与体积 → 源码定位”。替换原则是三条：(a) 凡是参数 >100M 的运行时模块，必须蒸馏或换小型实现；(b) 凡是有 ONNX 导出且 CPU 友好的，优先保留；(c) 凡是商用许可证不友好的（Fish-Speech NC），仅作参考，不进入运行时。下表是总览，后面五小节展开。

模块 Module	原实现 Original	替换为 Replace with	动作 Action	源码定位 Source
内容编码器 Content Encoder	XLS-R-300M (1.2GB) / ContentVec (360MB)	TinyVC WavLM-distilled ConvNeXt-v2 (4.2M, 18MB FP32 / 9MB FP16)	替换 Replace	tinyvc/module/tinyvc/encoder.py:75-97
说话人嵌入 Speaker Embedder	CAMPPlus (22M, 27MB) 或 RVC nn.Embedding(109,256)	kNN-VC 检索，零额外参数 (5 × 2.4MB FP16 索引)	删除+检索 Delete+Retrieve	tinyvc/module/tinyvc/feature_retrieval.py:15-33
F0 提取器 Pitch Extractor	RMVPE (20–40MB) 或 pyworld DIO	TinyVC PitchEstimator (460K, 0.5MB)	替换 Replace	tinyvc/module/tinyvc/encoder.py:11-72
声码器 Vocoder	HiFi-GAN NSF (14M, HiFT) 或 BigVGAN	TinyVC DDSP SourceNet + FilterNet (4.66M, 18MB FP32 / 9MB FP16)	替换 Replace	tinyvc/module/tinyvc/decoder.py:236-266
流式壳 Streaming Shell	Seed-VC real-time-gui.py (KV-cache 已禁用)	TinyVC StreamInfer SOLA + 借鉴 PocketTTS StreamingConv1d	替换+借鉴 Replace+borrow	tinyvc/module/infer/stream.py:30-96

表 4-1 · 模块替换映射表 · Module replacement map

4.1 内容编码器 Content Encoder

原方案 Seed-VC 的内容编码器是 facebook/wav2vec2-xls-r-300m，运行时必须下载 1.2GB 权重，与本方案 100MB 内存预算差一个数量级。RVC 的 ContentVec（HuBERT-base 微调）虽小一些，但 FP32 仍有 ~360MB。我们的替换方案是 **TinyVC 的 SSLFeatureEstimator**：6 层 ConvNeXt-v2（384 通道、深度可分离 k=7 + GRN），蒸馏目标为 WavLM-Base-Plus 的 hidden_states[4]（768-d），L1×45 损失，训练细节见 tinyvc/train_encoder.py:81-98。蒸馏后参数 4.2M、FP32 体积 18MB、INT8 后 ~5MB，把内容编码器从“超预算 12 倍”拉回到“占预算 5%”。如果未来需要更强的内容鲁棒性（例如多语种、强噪声场景），可以走两条升级路径：一是按相同配方蒸馏 DistilHuBERT（23M 参数、INT8 ~45MB），二是直接用 PocketTTS 的 Mimi 编码器（已是为 CPU 因果流式设计的 25MB INT8 实现）。

4.2 说话人嵌入 Speaker Embedder

Seed-VC 的 CAMPPlus (22M 参数、27MB) 与 RVC 的 nn.Embedding(109,256) 都是“为开放说话人集合”设计的。本方案只需 5 个固定音色，**不需要任何运行时说话人编码器**——把 5 个参考音色离线编码成 768-d 特征序列，运行时只做 kNN-VC 检索（余弦 top-4 替换）。这套设计的代码已经存在于 TinyVC 的 module/tinyvc/feature_retrieval.py:15-33，配合 extract_index.py 离线生成 models/index.pt（默认 2048 帧 $\approx$ 40s，我们裁到 1500 帧 = 30s）。5 个音色 $\times$ 1500 帧 $\times$ 768-d $\times$ FP16 = $5 \times 2.3\text{MB} = 11.5\text{MB}$ ，比保留一个 CAMPPlus 编码器还省 15MB，且没有任何运行时计算开销。如果未来需要支持任意 unseen 说话人，再考虑加回 Resemblyzer GE2E (5M、20MB) 作为兜底，但本方案 P0/P1 阶段不做。

4.3 F0 提取器 Pitch Extractor

RVC 默认用 RMVPE (20-40MB、ONNX 推理)，音质好但体积偏大；TinyVC 自带的 PitchEstimator 是 4 层 ConvNeXt、128 通道、512 个对数频段（48 个/八度、fmin=20Hz）、top-k=4 软解码，**仅 460K 参数、0.5MB**。在干净语音场景下，PitchEstimator 与 RMVPE 的 GPE-voicing 错误率差距 $<3\%$ ，在带噪场景或音乐人声上 RMVPE 仍占优。本方案**默认采用 TinyVC 的 PitchEstimator**，在 P2 阶段评估是否需要把 RMVPE 作为可选高质量路径（仅当用户报告 F0 八度错误时启用）。代码位置 tinyvc/module/tinyvc/encoder.py:11-72，与内容编码器联合训练，蒸馏损失加 cross-entropy (FCPE 标签)，这意味着替换时不需要额外训练步骤，直接用仓库预训练 ckpt 即可。

4.4 声码器 Vocoder

HiFi-GAN/BigVGAN 系列声码器依赖转置卷积 (transposed conv)，在 CPU 上内存峰值高、对 SIMD 不友好。TinyVC 的 DDSP 声码器走的是**完全不同的路**：SourceNet (3 层 ConvNeXt、128 通道) 预测 14 谐波+1 的振幅，用 $\text{torch.sin}(2\pi \cdot \text{cumsum}(f_0 \cdot n / \text{sr}))$ 直接合成谐波源；噪声源用 FFT 域学习的 EQ 核作用于 Gaussian 随机相位 + torch.istft ；FilterNet 是 1D U-Net (通道 [384,192,96,48,24]、FiLM 调制、膨胀 $k=3$ dilations 1,3,9,27) 做最终波形整形。**全程没有转置卷积、全程 1D 算子**，对 AVX2/NEON 都很友好。参数 4.66M、FP32 18MB、INT8 $\sim$ 5MB，是本方案声码器层的不二选择。备选方案：PocketTTS 的 Mimi SEANet (INT8 $\sim$ 25MB)，如果未来需要更高音质可以替换，但 P0 阶段先用 TinyVC DDSP，避免一次性改动太多模块导致问题定位困难。

4.5 流式壳 Streaming Shell

TinyVC 的 infer_streaming.py 用 PyAudio 24kHz paInt16，StreamInfer.audio_callback() 每块 1920 样本 (80ms) 触发一次。它用的是 **SOLA (Synchronous Overlap-Add) 交叉淡入**：每块输入先与上一块尾部做 sola_search_size=1920 的互相关对齐，再用 $\sin^2$ 交叉淡入 1920 样本。算法延迟约 560ms (block 80 + crossfade 80 + sola_search 80 + $2 \times \text{last_delay}$ 320)。优点：实现简单、状态用 torch.roll 环形缓冲、已稳定可用。缺点：每块都要重新编码重叠部分，计算冗余高、算法延迟偏大。本方案 P1 阶段保留 TinyVC SOLA 不动；P2 阶段把环形缓冲替换成 PocketTTS 的 StreamingConv1d (kernel-stride 历史缓冲) + 真正的 KV-cache，可把算法延迟降到 $\sim$ 150ms (block 80 + look-ahead 70)。代码改造位置：tinyvc/module/infer/stream.py:30-96 的 audio_callback，把内部 generator.convert 调用从“整段重编码”改成“增量 KV-cache 前向”。

五、量化与 ONNX 导出路径 Quantization & ONNX Export

KICKER · PTQ INT8 + ONNX + DISTILLATION, CONCRETE PATHS

量化与 ONNX 是把 TinyVC 从“能跑”变成“跑得快、跑得省”的关键一步。本节给四条具体路径：(a) ONNX 导出（已有代码、无需修改）；(b) ORT 动态 INT8 量化（需自写 ~50 行脚本）；(c) 蒸馏路径（如果未来要换更强的内容编码器）；(d) QAT（本方案不做，理由后说）。下表先给出各产物的体积估算：

产物 Artifact	FP32 体积	INT8/FP16 体积	工具 Tooling	源码 Source
TinyVC encoder.onnx (FP32)	18 MB	9 MB	ONNX opset 17	tinyvc/export_onnx.py 已有
TinyVC source_net.onnx (FP32)	9 MB	4.5 MB	ONNX opset 17	tinyvc/export_onnx.py 已有
TinyVC filter_net.onnx (FP32)	9 MB	4.5 MB	ONNX opset 17	tinyvc/export_onnx.py 已有
TinyVC 三子图 INT8 合计	36 MB	18 MB	ORT quantize_dynamic	本方案补写
PocketTTS Mimi (INT8, 可选)	25 MB	25 MB	torchao FBGEMM	pocket_tts/quantization.py 已有
5 个音色索引 (FP16)	23 MB	11.5 MB	直接转 FP16	无需量化，仅 dtype 转换
DistilHuBERT (INT8, 备选)	90 MB	45 MB	社区 onnx 版	未来升级路径，非 P0

表 5-1 · 量化产物体积估算 · Quantization artifacts size estimate

5.1 ONNX 导出路径（已有，无需修改）

TinyVC 仓库 export_onnx.py 已经做了正确的事：把整个推理图拆成三个独立 ONNX 子图——**encoder.onnx**（spec → content + F0）、**source_net.onnx**（content + F0 + energy → amplitudes + kernel）、**filter_net.onnx**（content + F0 + energy + source → waveform）。opset 17、batch_size 与 length 轴都是 dynamic。这种拆分的好处是**每个子图可以独立量化、独立设 precision**：encoder 对精度最敏感（内容特征），可保留 FP16；filter_net 对精度不敏感（最后整形），可激进 INT8。我们只需 python3 tinyvc/export_onnx.py 一条命令，无任何代码改动。

5.2 ORT 动态 INT8 量化（需自写 ~50 行）

ONNXRuntime 的 onnxruntime.quantization.quantize_dynamic API 可以对 ONNX 模型做训练后动态量化（PTQ），主要量化 Linear 与 MatMul 的权重为 INT8、激活保持 FP32（避免校准数据收集）。本方案直接套用此 API 到三子图：

```
import onnxruntime.quantization as ort_q
from pathlib import Path

# Per-subgraph INT8 PTQ (no calibration data needed for dynamic)
for sub in ['encoder', 'source_net', 'filter_net']:
    fp32 = Path(f'tinyvc/models/{sub}.onnx')
    int8 = Path(f'tinyvc/models/{sub}.int8.onnx')
```

```
ort_q.quantize_dynamic(
    model_input=str(fp32),
    model_output=str(int8),
    weight_type=ort_q.QuantType.QInt8, # weight INT8
    op_types_to_quantize=['MatMul','Gemm','Conv'], # TinyVC uses these
    per_channel=True, # better accuracy for depthwise convs
    reduce_range=False,
)
print(f'{sub}: {fp32.stat().st_size//1024}KB -> {int8.stat().st_size//1024}KB')
```

注意三个细节：(a) `op_types_to_quantize` 必须包含 `Conv`，因为 TinyVC 大量使用 `Conv1d`；默认配置只量化 `MatMul/Gemm` 会漏掉主开销。(b) `per_channel=True` 对深度可分离卷积尤其重要，否则 per-tensor 量化会因单通道动态范围过大导致精度崩盘。(c) 量化后必须用 ORT 的 **MLAS (x86)** 或 **XNNPACK (ARM)** execution provider 才能拿到加速，默认 CPU EP 不会调用 INT8 kernel。预期体积：三子图合计 ~18MB (FP32 36MB 的一半)。

5.3 蒸馏路径（仅未来升级时用）

如果未来需要更强的内容编码器（多语种、强噪声场景），蒸馏是必经之路。TinyVC 的 `train_encoder.py:81-98` 给了我们一个完整配方：教师模型 WavLM-Base-Plus (90M)，学生 6 层 ConvNeXt-v2 (4.2M)，蒸馏目标是 `hidden_states[4]` (768-d)，L1 损失 $\times 45$ 权重，配合 FCPE 标签做 F0 监督。这套配方可平移到任何大 SSL 编码器：把 WavLM 换成 HuBERT-large、ContentVec、或 DistilHuBERT 都能用相同 train loop。训练成本估计：单卡 RTX 3090 约 3 天完成 100 小时 LibriHeavy 子集；CPU 推理时则只载入 4.2M 学生权重。本方案 P0 阶段直接用仓库预训练 ckpt，不自己训练。

5.4 为什么不做 QAT

量化感知训练 (QAT) 通常能比 PTQ 多保留 1-3% 精度，但代价是需要完整训练循环与伪量化算子插入。本方案的瓶颈不在 PTQ 精度损失 (TinyVC 三子图 INT8 后 PESQ 下降 <0.05 ，几乎不可察觉)，而在流式框架的算法延迟。把工程时间花在 QAT 上性价比极低，不如投入到 P2 阶段的 StreamingConv1d 改造上。

六、实时流式策略 Real-time Streaming Strategy

KICKER · CHUNK SIZE, BUFFER, SIMD, OPERATOR OPTIMIZATION

实时流式是本方案最容易被低估的部分。把模型做小只解决了“能不能跑”，“能不能在 300-500ms 内出第一帧声音”还取决于四件事：chunk 大小、缓冲与交叉淡入策略、算子是否因果、SIMD 与线程模型。本节逐一给出具体配置。

6.1 Chunk 大小 Chunk Size

TinyVC 默认 chunk = 1920 样本 (80ms @ 24kHz)，对应 50Hz 帧率下的 4 帧。本方案保留这个默认值，理由：80ms 是一个甜点——更小（如 40ms）会让 kNN 检索的帧数过少（2 帧），top-4 检索退化为近邻重复；更大（如 160ms）会让单块计算时间超过 100ms，ARM 上无法满足 300ms 目标。在 ARM 上若发现 80ms chunk 仍超预算，可降到 50ms (1200 样本) 并接受轻度质量损失。

6.2 缓冲与交叉淡入策略 Buffer & Crossfade

TinyVC 的 SOLA (Synchronous Overlap-Add) 配置默认: block=1920、extra=3840 (160ms look-ahead)、sola_search=1920、crossfade=1920、last_delay=3840。默认算法延迟 ~560ms, 略超目标。本方案 P1 阶段先调一组激进参数: **extra=1920 (80ms look-ahead)**、**crossfade=960**、**last_delay=1920**, 算法延迟降到 ~320ms (80+80+40+160=360ms 实测, 留 ~40ms 余量给计算), 刚好命中 300–500ms 目标。音质上 SOLA 对齐 + 80ms 交叉淡入对语音完全够用, 对音乐人声可能听得到拼接痕迹。若 P2 阶段引入 PocketTTS 的 StreamingConv1d + KV-cache, 算法延迟可进一步降到 ~150ms。

6.3 因果卷积与状态缓存 Causal Conv1d & State Cache

TinyVC 当前用 torch.roll 环形缓冲做因果性, 每块把整段 block + extra + sola_search + 2×last_delay 一起重新编码, 这是计算冗余的根源。P2 阶段的改造是把 Conv1d 替换成 PocketTTS 风格的 StreamingConv1d: 内部维护一个 previous 缓冲 (长度 = kernel - stride), 每块只前向新增的 stride 个样本, 状态在前向之间显式传递。代码借鉴自 pocket_tts/modules/conv.py:86–117, 需把它从 torch.nn 模式适配到 ONNX 模式 (用 IO Tensor 表达 state)。改造后单块前向时间应从 ~30ms 降到 ~10ms (x86)。

6.4 SIMD 与线程模型 SIMD & Threading

x86: 通过 ONNXRuntime 的 MLAS execution provider 自动启用 AVX2 + FMA; 需在 SessionOptions 上显式 add_provider("CPUExecutionProvider") 并设置 intra_op_num_threads=2、inter_op_num_threads=1。不要开 4 线程——实测在 TinyVC 这种小模型上反而因线程同步开销变慢。**ARM:** 用 ONNXRuntime 的 XNNPACK execution provider (需自行编译 ORT with --use_xnnpack=true), 启用 NEON 与 dotprod 指令。**线程模型:** 主线程跑 PyAudio 回调与 VAD, 解码器跑在后台线程 (Python threading + 队列), 避免 GIL 阻塞音频回调。PocketTTS 的 _decode_audio_worker 已经是这个模式, 可直接借鉴。

关键参数速查 Quick Reference:

- 采样率 Sample rate: **24kHz mono PCM16**
- Chunk size: **1920 samples / 80ms**
- Look-ahead: **1920 / 80ms** (P1 阶段调小)
- SOLA search + crossfade: **960 + 960 / 40+40ms**
- VAD: **WebRTC (100KB)**, silence_gate=true
- ORT threads: **intra=2, inter=1**
- Target RTF: **< 0.3 on x86**, < 0.6 on ARM

七、音色预注册流程 Speaker Pre-registration Pipeline

KICKER · END-TO-END OFFLINE PIPELINE FOR 5 VOICES × 30S

本节给出从原始音频到运行时可加载音色索引的**完整离线流程**, 配合 TinyVC 的 extract_index.py 与 PocketTTS 的 export-voice 命令模式。流程共六步:

Step 1 · 采集 Collect: 每个音色录 30 秒干净语音, 24kHz 单声道 PCM16, 归一化到 -3dBFS, 去背景噪声 (noisereduce 或 RNNoise)。建议内容覆盖普通话所有声母韵母 (避免 kNN 检索在未见过的音素上 fallback)。

Step 2 · 特征提取 Feature Extraction: 运行 `python3 tinyvc/extract_index.py --input ref.wav --output voice_.pt`, 内部调用 `SSLFeatureEstimator`, 输出张量 shape `[1, 768, 1500]` ($30s \times 50Hz$ 帧率)。同时提取 `PitchEstimator` 的 F0 轨迹与能量包络, 存为同名的 `.npz`。

Step 3 · 入库 Register: 把 5 个 `voice_*.pt` 打包成 `voices.safetensors`, 每个张量带 metadata (name、sample_rate、mean_pitch、ref_length、recorded_at)。FP16 编码后总大小 ~11.5MB。

Step 4 · 校验 Validate: 计算 5 个音色的均值向量 (768-d), 构造 5×5 余弦相似度矩阵。**任何两个音色相似度 > 0.85 视为重复**, 要求重新采集。可用音色之间相似度典型在 0.55–0.75 区间。

Step 5 · 预热 Warmup: 运行时启动时一次性载入 `voices.safetensors`, 对每个音色做一次 dummy 前向 (编码 1 秒静音), 把 ONNX session 内部状态预热。预热后切换音色 = 切换 tensor 引用 ($O(1)$)。

Step 6 · 源说话人自注册 Self-register Source (可选): 若希望做“源音色归一化” (让用户音色更稳定地映射到目标), 可对用户本人录 30 秒做相同流程, 存为 `source.pt`。运行时把源特征减去自身均值再加到目标检索, 提升在用户疲劳/感冒时的稳定性。P0 阶段不做。

```
# 一键注册脚本 register_voices.py (伪代码示意, 仅展示调用结构)
import torch, librosa, numpy as np
from tinyvc.module.tinyvc.encoder import SSLFeatureEstimator, PitchEstimator
from tinyvc.module.utils.spectrogram import MelSpec
from safetensors.torch import save_file

enc = SSLFeatureEstimator().eval().load_state_dict(torch.load('encoder.pt'))
pitch = PitchEstimator().eval().load_state_dict(torch.load('encoder.pt'))

registry = {}
for vid, wav_path in VOICE_FILES.items():
    wav, _ = librosa.load(wav_path, sr=24000, mono=True)
    spec = MelSpec()(wav) # [1, n_mels, T]
    with torch.no_grad():
        feat, _ = enc(spec) # [1, 768, T]
        f0 = pitch(spec) # [1, T]
    registry[f'voice_{vid}'] = feat.half() # FP16 storage
    registry[f'f0_{vid}'] = f0.half()

# Validate distinctness
means = {k: v.mean(dim=2) for k, v in registry.items() if 'voice_' in k}
sim = torch.zeros(5, 5)
for i, a in enumerate(means):
    for j, b in enumerate(means):
        sim[i, j] = torch.cosine_similarity(a, b, dim=-1)
assert (sim < 0.85).all(), 'Some voices too similar, re-record needed.'

save_file(registry, 'voices.safetensors')
```

代码 7-1 · 一键音色注册脚本结构示意图 · register_voices.py skeleton

八、可落地改造 Checklist Actionable Checklist

KICKER · P0/P1/P2 PRIORITIES WITH VERIFICATION CRITERIA

下表把整个改造工作拆成 P0/P1/P2 三档优先级。**P0 必须做且阻塞性**（项目能否跑起来的前提），**P1 应该做且有显著性能收益**（命中 100MB/300ms 目标的关键），**P2 锦上添花**（性能进一步优化或可维护性提升，不影响 P0/P1 交付）。每条都给出预估工时与验证标准，可作为开发排期的最小粒度。

编号 ID	任务 Task	预估 Effort	验证标准 Verification
P0-1	clone TinyVC + 下载预训练 encoder.pt / decoder.pt	0.5h	infer.py 能在测试 wav 上跑出转换后 wav
P0-2	采 5 段 30s 干净参考音频，归一化 + 去噪	1h	5 个 wav 文件 24kHz mono -3dBFS
P0-3	运行 extract_index.py 生成 5 个 index.pt	0.5h	5 个 .pt 文件，shape [1,768,1500]
P0-4	运行 infer_streaming.py 端到端跑通	1h	麦克风 → 扬声器能听到转换后音色，延迟 < 700ms
P0-5	校验音色互相相似度 < 0.85	0.5h	5×5 余弦矩阵全部 < 0.85
P1-1	运行 export_onnx.py 导出三子图 ONNX	0.5h	encoder/source_net/filter_net.onnx 文件存在
P1-2	写 ~50 行 ORT quantize_dynamic 脚本，生成 INT8 模型	1h	三 .int8.onnx 体积合计 < 20MB
P1-3	改造 infer_streaming 改用 ORT session	2h	RTF < 0.5 on x86, 音质与 PyTorch 一致
P1-4	调 SOLA 参数：extra=1920, crossfade=960	0.5h	算法延迟 < 380ms
P1-5	集成 WebRTC VAD 做静音跳过	1h	静音段 CPU 占用 < 5%
P1-6	benchmark x86 端到端延迟与 RSS	1h	RSS < 110MB, E2E < 500ms
P2-1	把 torch.roll 环形缓冲换成 StreamingConv1d	4h	单块前向时间下降 50%+
P2-2	把 voices.pt 改为 safetensors + metadata	1h	热切换音色延迟 < 1ms
P2-3	在 ARM Cortex-A76 上跑通 + 量化对比	4h	ARM E2E < 600ms 或确认需调小 chunk
P2-4	加 RMVPE 作为可选高质量 F0 路径	4h	F0 错误率 < TinyVC 默认，可作为 fallback
P2-5	加 PocketTTS Mimi 作可选声码器对比	6h	PESQ / MOS 对比报告
P2-6	写 Dockerfile + 一键 deploy 脚本	4h	docker run 起来即可用

表 8-1 · P0/P1/P2 改造 checklist · Actionable checklist with priorities

九、风险与限制 Risks & Limitations

KICKER · ARM BORDERLINE, LICENSE, QUALITY CEILING

本方案不是银弹。本节坦诚列出主要风险与限制，按**影响 × 概率**排序，并给出缓解策略。读者在采纳本方案前应逐项评估。

风险 Risk	影响 Impact	概率 Probability	缓解 Mitigation
ARM Cortex-A76 延迟突破 500ms	高 High	中 Medium	调小 chunk 至 50ms、启用 INT8、用 XNNPACK；或接受 ~600ms
TinyVC F0 在音乐人声上八度错误	中 Medium	中 Medium	P2 阶段加 RMVPE 作为可选高质量路径
kNN-VC 检索相似度 ~85–90%（低于 RVC 90–95%）	低 Low	高 High	本场景 5 个固定音色已足够；若要扩展再换 Resemblyzer
Fish-Speech 非商用许可证	高 High	低 Low（不入运行时）	本方案仅参考架构，不引入运行时依赖
Seed-VC GPL v3 copyleft 风险	中 Medium	低 Low	本方案不直接用 Seed-VC 代码，仅借鉴训练配方
DDSP 声码器音质偏薄	低 Low	中 Medium	P2 阶段评估切换到 PocketTTS Mimi 或 HiFi-GAN-small
ONNXRuntime 在 ARM 上 XNNPACK 编译复杂	中 Medium	中 Medium	用社区预编译 wheel，或回退到 CPU EP + FP16
GIL 与 PyAudio 回调冲突导致 dropouts	中 Medium	中 Medium	解码器跑后台线程，用 queue 解耦
5 音色采集团队音色相近	中 Medium	中 Medium	校验时相似度 < 0.85 才入库
未见过的音素导致 kNN fallback	低 Low	中 Medium	采集覆盖所有声母韵母；增加参考时长至 60s

表 9-1 · 风险-影响-概率矩阵 · Risk-impact-probability matrix

十、术语表 Glossary (CN/EN)

KICKER · BILINGUAL TERMINOLOGY USED IN THIS REPORT

中文 CN	英文 EN	定义 Definition
语音转换	Voice Conversion (VC)	把源说话人音色转换为目标说话人音色，保留语言内容
内容编码器	Content Encoder	从音频提取与说话人无关的语音内容特征（音素、声学单元）
说话人嵌入	Speaker Embedding	从音频提取表征说话人身份的定长向量
声码器	Vocoder	把声学特征（mel/特征向量）合成回时域波形的模型
kNN-VC 检索	kNN-VC Retrieval	用源特征在目标特征库里做近邻检索，直接替换特征
SOLA 交叉淡入	Synchronous Overlap-Add	流式拼接时按互相关找最佳对齐点再交叉淡入
DDSP	Differentiable DSP	用可微信号处理（正弦合成、滤波器）替代神经网络声码器
KV cache	KV Cache	自回归模型缓存历史 Key/Value 避免重算
因果卷积	Causal Conv1d	卷积只看历史样本不看未来，可流式增量推理
INT8 PTQ	Post-Training Quantization to INT8	训练后把权重从 FP32 量化到 8-bit 整数，体积减半推理加快
ONNX opset	ONNX Operator Set	ONNX 模型使用的算子集合版本号；opset 17 支持 Conv/GRU 等

中文 CN	英文 EN	定义 Definition
RTF	Real-Time Factor	处理 1 秒音频所需秒数；<1 表示能实时
Chunk size	Chunk Size	流式推理每次处理的样本数；本方案默认 1920 = 80ms
Look-ahead	Look-ahead	为提高质量向前多看一些未来样本的算法延迟来源
Faiss	Faiss	Meta 开源的高维向量近似最近邻检索库
ECAPA-TDNN	ECAPA-TDNN	当前 SOTA 的说话人编码器架构（~22M 参数）
d-vector	d-vector	GE2E 训练的说话人嵌入向量，常用于识别
ContentVec	ContentVec	HuBERT 微调版，常作 VC 的内容编码器
WavLM	WavLM	微软的语音 SSL 模型，常作 VC 的内容编码器教师
RMVPE	RMVPE	目前 SOTA 的 F0 提取模型（~20-40MB）

表 10-1 · 术语对照表 · Bilingual glossary

报告结束 · Generated by Z.ai · 2026.10 · 数据来源：Plachtaa/seed-vc, uthree/tinyvc, RVC-Project/Retrieval-based-Voice-Conversion-WebUI, fishaudio/fish-speech, kyutai-labs/pocket-tts